

Tema1:

Configuración de un proyecto web y Operaciones en CRUD generando una Clase en un archivo de migración con la instalación de paquetes para la correcta ejecución en JavaScript de los métodos GET, POST, PUT, DELETE

¿Qué son GET, POST, PUT, ¿DELETE?

Estos son verbos HTTP, usados en aplicaciones web y APIs

CRUD

describe operaciones sobre datos (normalmente en una base de datos):

Create → Crear

Read → Leer

Update → Actualizar

Delete → Eliminar

<https://theharbour.ltd/product/exportacion-del-modelo-y-del-codigo-en-github/>

Contenido

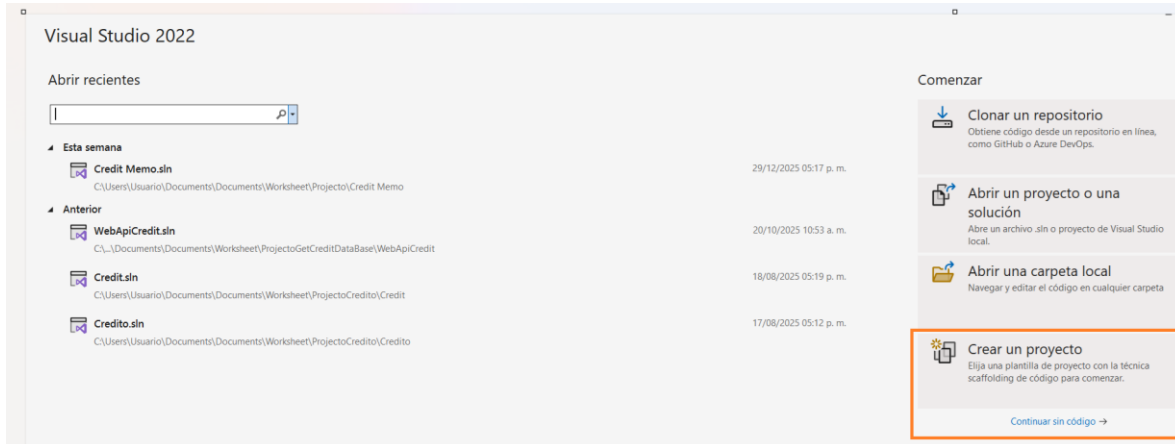
Capacitación Desarrollando aplicaciones con Entity Framework	2
Características de Instalación de la Página Web:.....	2
Filtro con lenguaje de programación C#	2
Seguimiento de pasos para realizar los las operaciones CRUD.....	2

Capacitación Desarrollando aplicaciones con Entity Framework

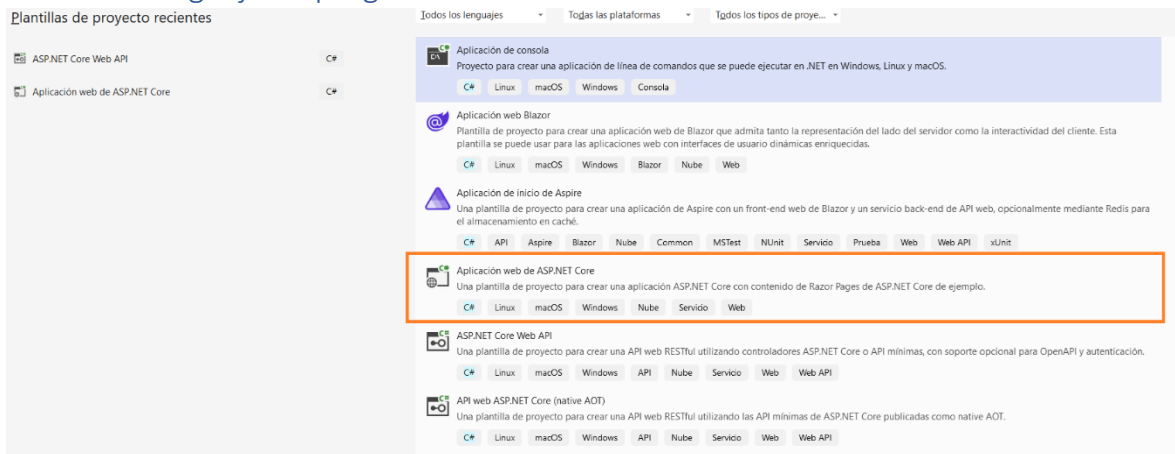
Características de Instalación de la Página Web:

<https://theharbour.ltd/2025/09/05/quieres-aprender-hacer-una-api/>

Se va a crear de un modelo de negocio, diferente es una Entrega de Pedido



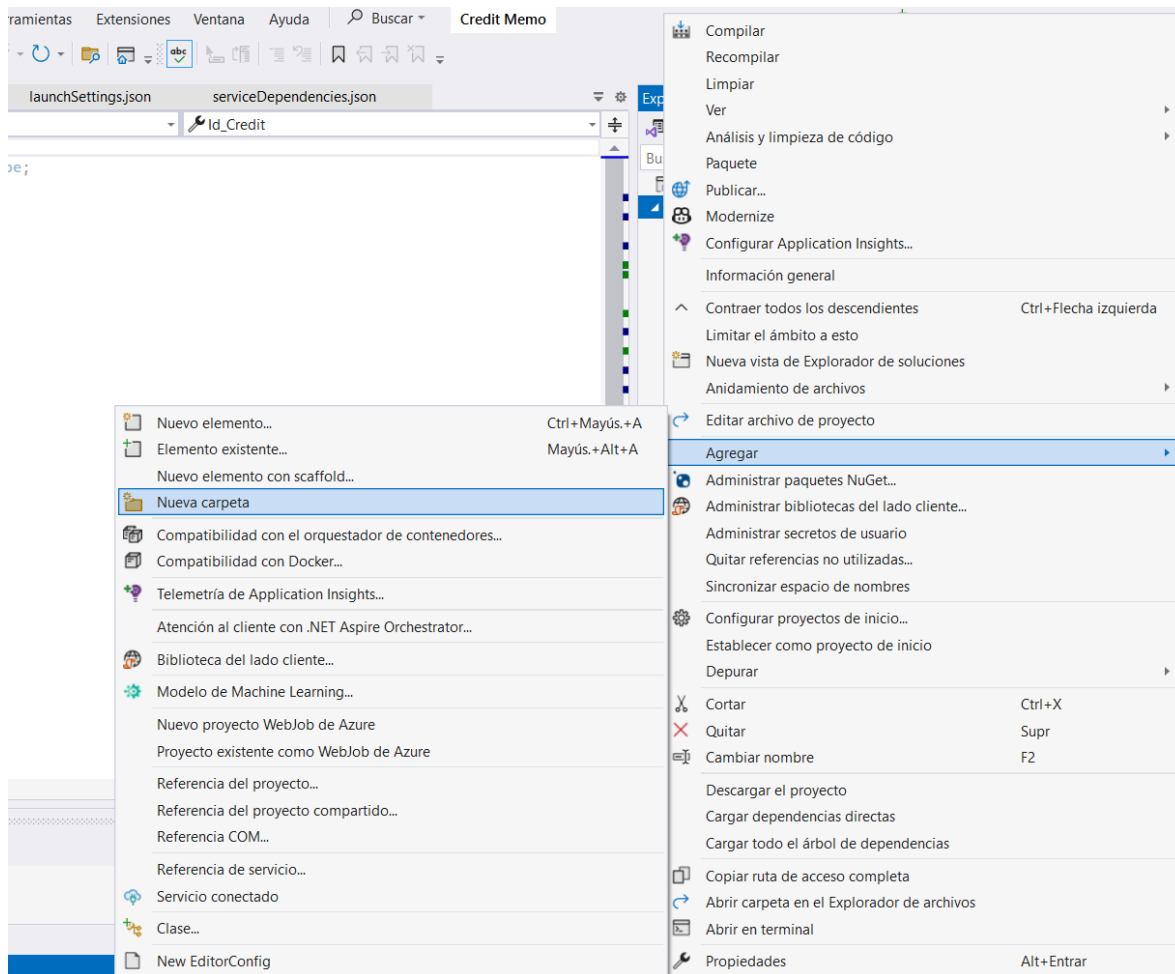
Filtro con lenguaje de programación C#



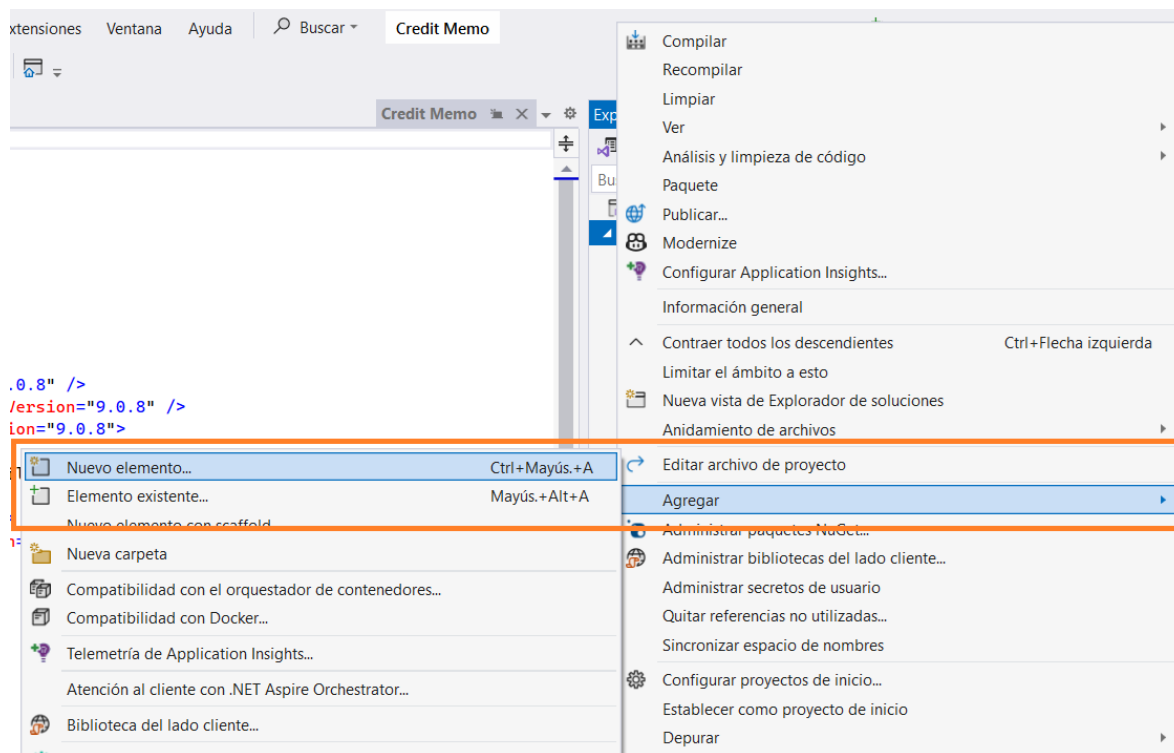
Seguimiento de pasos para realizar los las operaciones CRUD

1-Crea una carpeta **Models**, donde se va a crear una clase por la que se quiere ejecutar los métodos mostrar, actualizar, operaciones CRUD (Create, Read, Update, Delete).

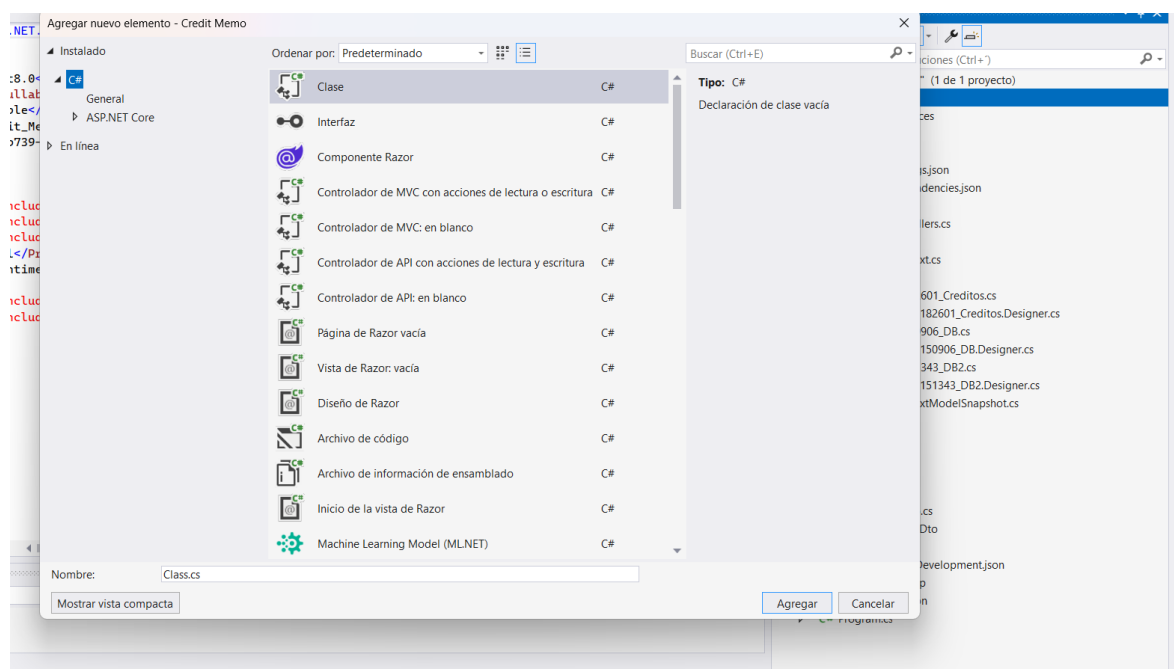
El ámbito(public) tipo de dato {get;set}

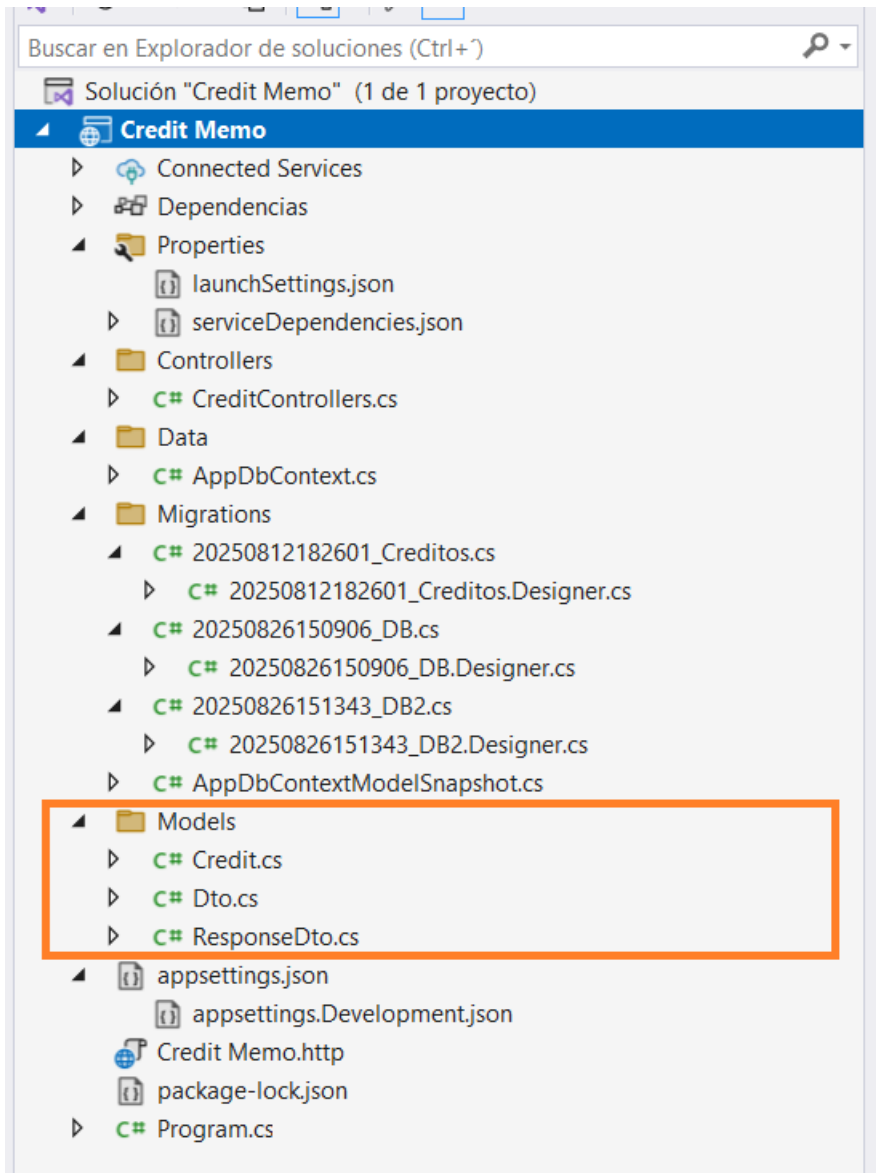


Agregando la clase, de la pantalla elemento existente



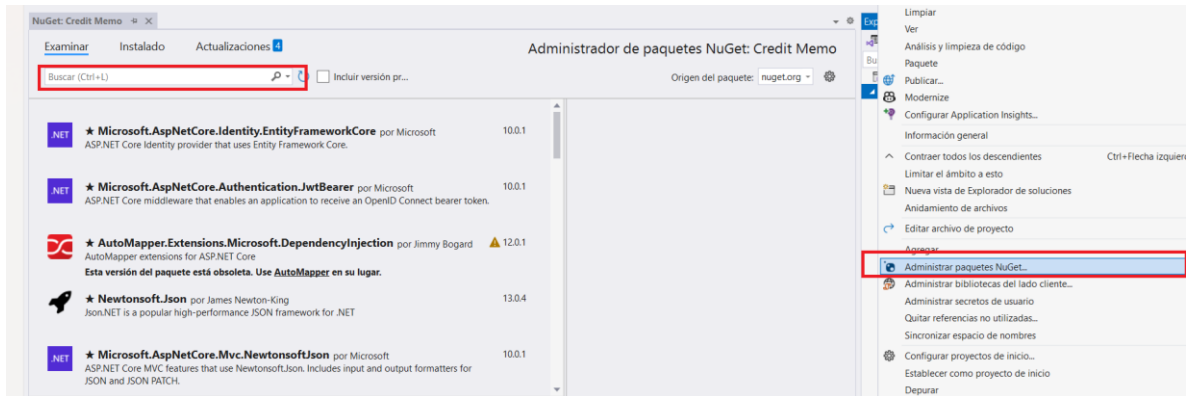
Seleccionamos Clase



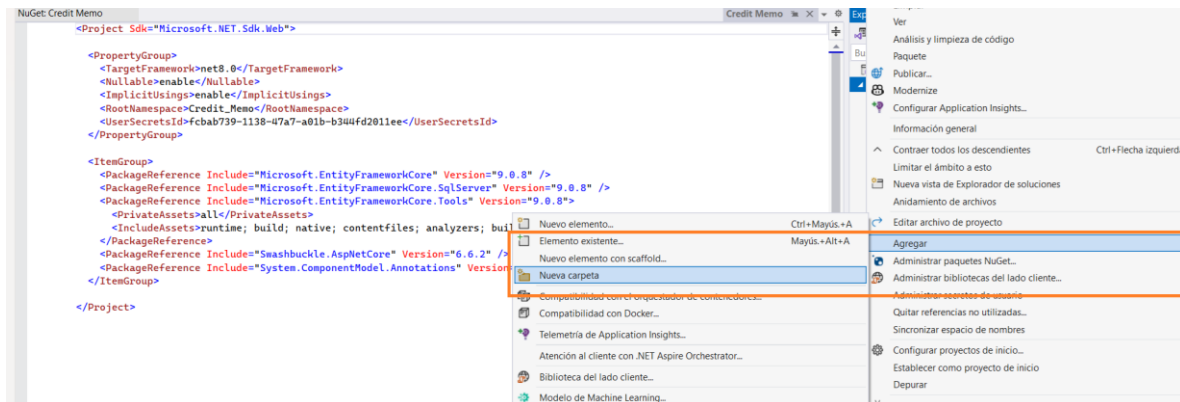


2-Buscar los paquetes Nuguets

1. MicrosoftEntityFrameworkCore
2. MicrosoftEntityFrameworkCore. Tools
3. MicrosoftEntityFrameworkCore.Sql



3- Agregar una nueva Clase, que se llame **AppDbContext**, el cual hereda de DbContext



DbContext es una clase de Entity Framework que:

- Maneja la conexión a la base de datos
- Permite consultar datos
- Permite insertar, actualizar y eliminar registros

```
public class AppDbContext:DbContext

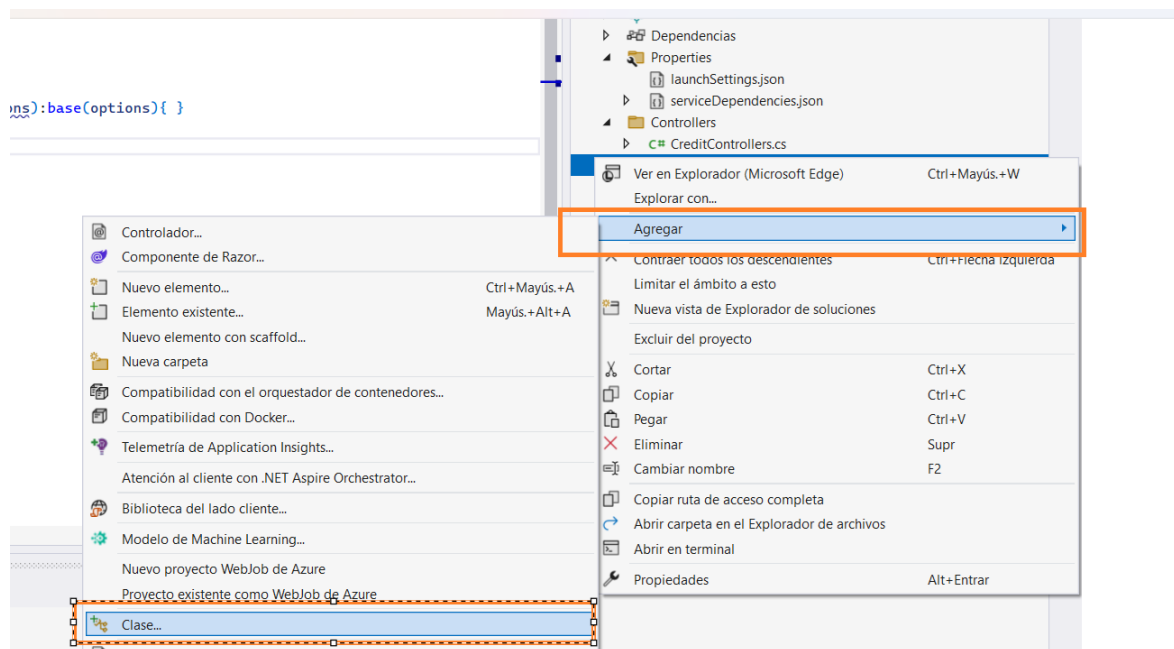
{

    public AppDbContext(DbContextOptions<AppDbContext>options):base(options){ }

    public DbSet<Credit> Credits { get; set; }

}
```

Dbset: representa a una tabla



4- Agregar otra **Clase Dto**: No tiene lógica comunica los datos a la Api, a través del formato Json.

-Por defecto, una clase es concreta, es decir, instanciable, a menos que explícitamente se marque como abstract.

-Las clases abstractas pueden contener miembros abstractos (que no tienen ninguna implementación y deben invalidarse en clases derivadas) y miembros totalmente implementados (como métodos normales, propiedades y constructores). Esto permite que las clases abstractas proporcionen funcionalidad común, a la vez que requieren clases derivadas para implementar miembros abstractos específicos.

Explicación de vacio:

Esto NO indica herencia, porque:

Dto es la clase base

Las clases base no declaran a quién heredan

Las clases hijas son las que dicen de quién heredan

5- Agregar otra **clase ResponseDto**, no sirve como respuesta a todas las solicitudes que enviemos. Es una **clase DTO (Data Transfer Object)**.

Los DTO se usan para **transportar datos** entre capas de una aplicación (por ejemplo, del backend al frontend en una API).

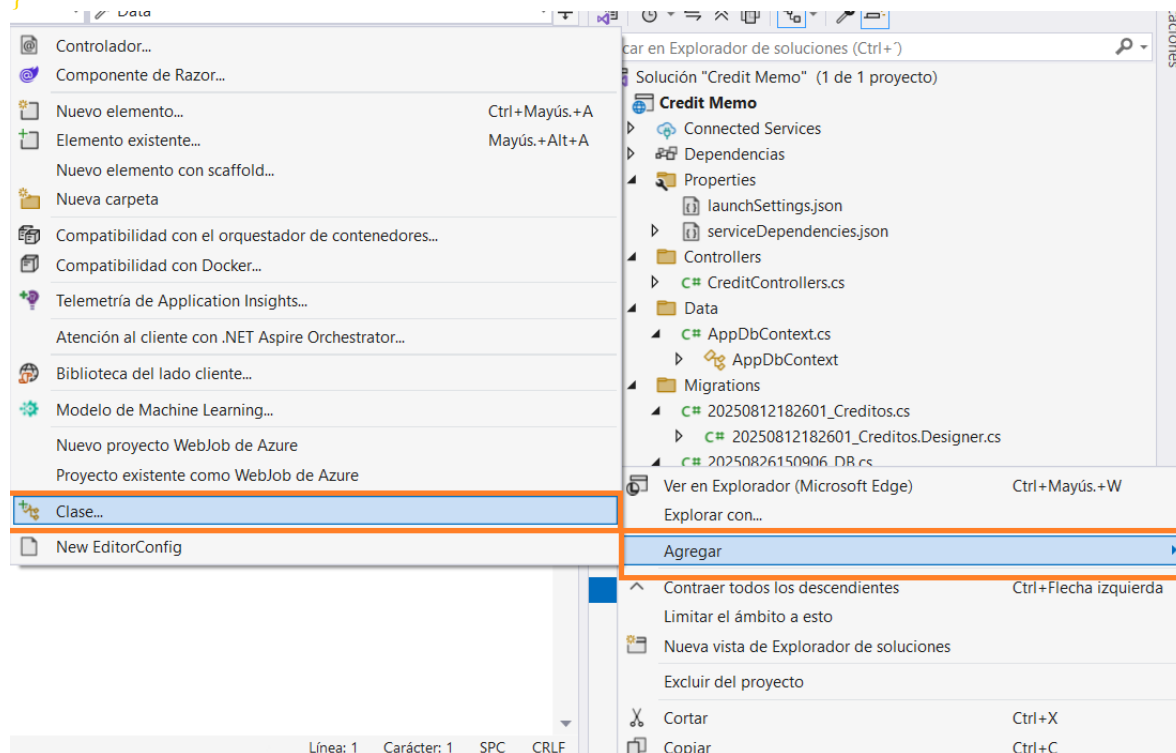
? indica que **puede ser null** (nullable reference types).

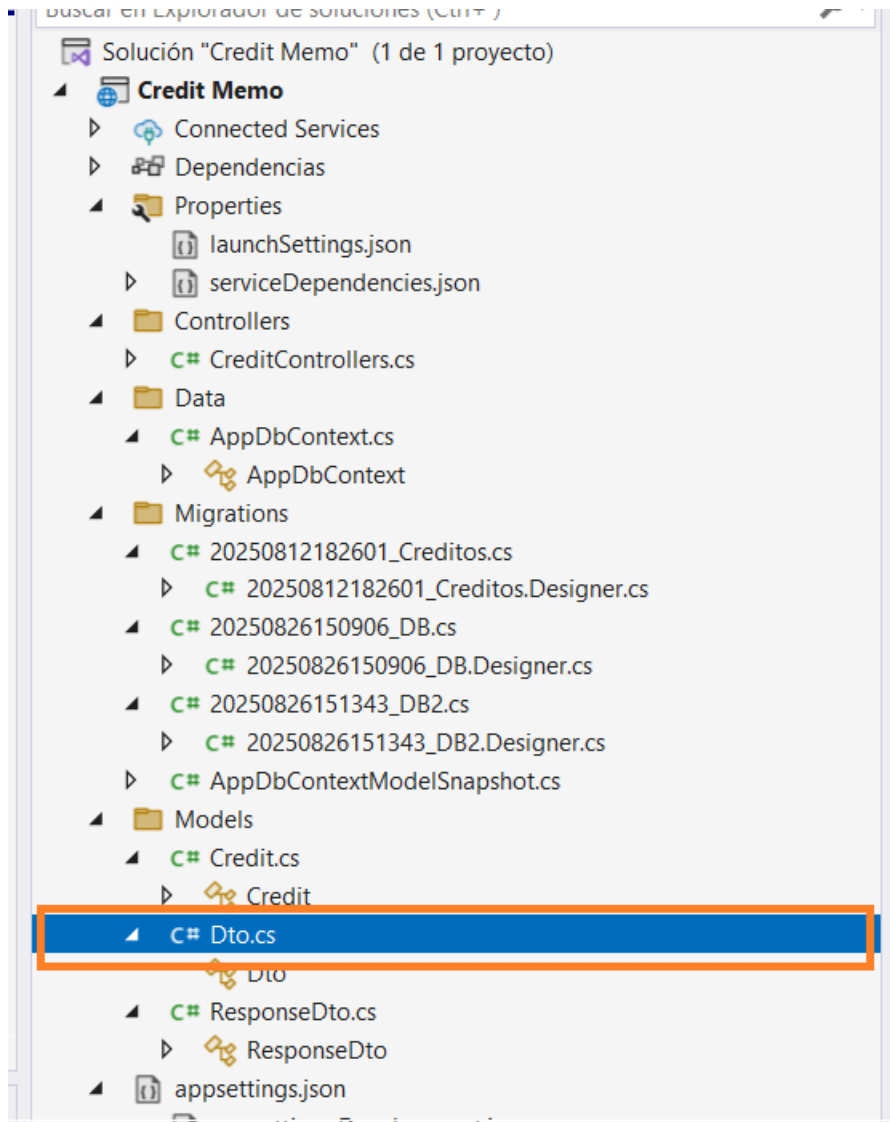
Data: contiene la información principal de la respuesta.

Tipo **object**: permite almacenar **cualquier tipo de dato** (lista, objeto, string, etc.).

`{ get; set; }` permite leer y asignar el valor.

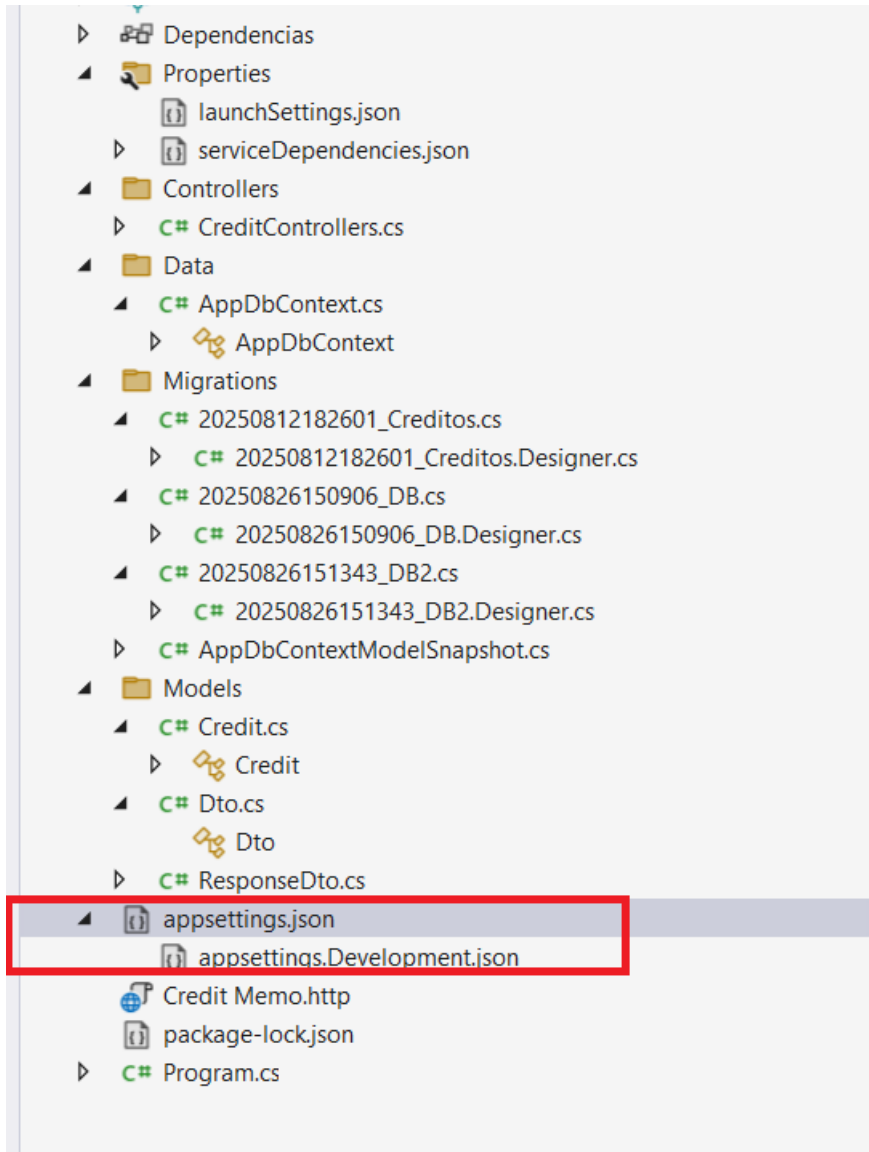
```
public class ResponseDto
{
    public object? Data { get; set; }
    public bool Success { get; set; } = true;
    public string Message { get; set; } = "";
}
```





```
public class Dto
{
}
```

6- Crear el string de conexión, nos vamos al appsettings.json, abro el Management studio, y copio la conexión del server name(DESKTOP-9QN799I)



```
"ConnectionStrings": {  
  "DefaultConnection": "Server=localhost;  
  DataBase=master;Trusted_Connection=True;TrustServerCertificate=True"},
```

7- Insertar el string de conexión en Programs.cs, en nuestro inyector de dependencia.

```
builder.Services.AddDbContext<AppDbContext>(options =>  
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

//options: Configura **qué motor de base de datos** se usará
//builder.Services (bases de datos, servicios, repositorios, etc.).
// GitHub solo hospeda el código, **no provee services** ni nada que se ejecute en tu aplicación.

8-Realizar la Migración, que se hace la consola de administración de paquetes.

Comandos

1- Add-MigrationDB,

2-Update-Database

Función: Genera un archivo de migración en la carpeta Migrations. Estructura de migración, estructura física

-Tablas

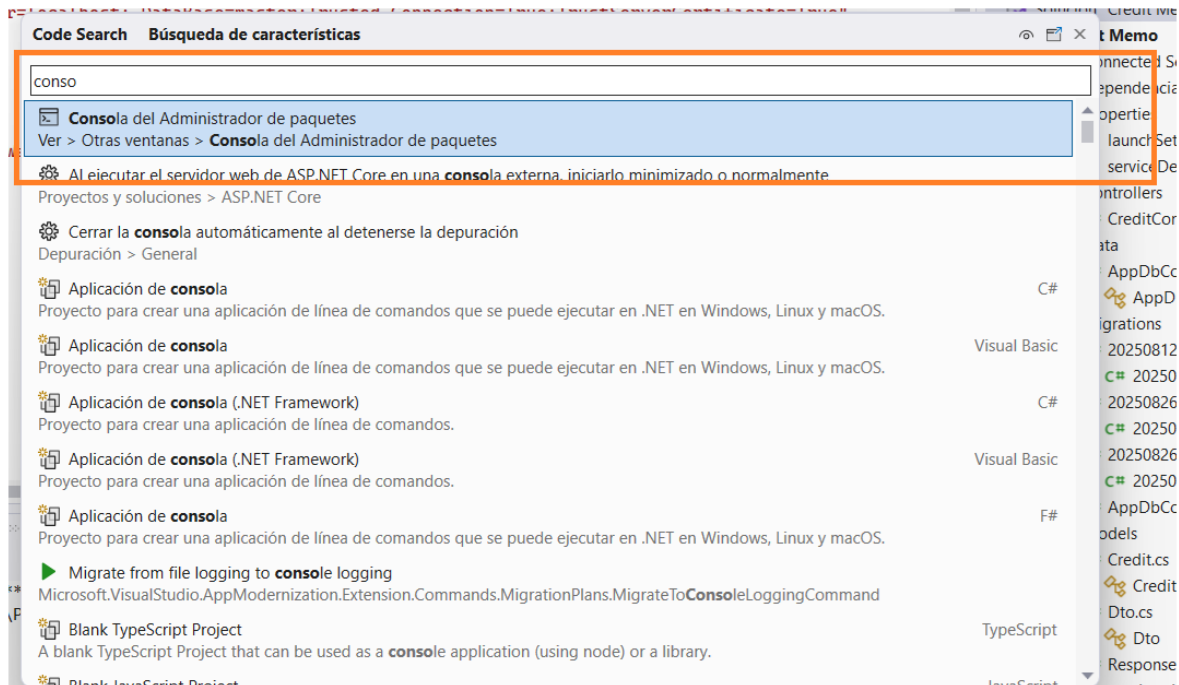
-Columnas

-Tipos de datos

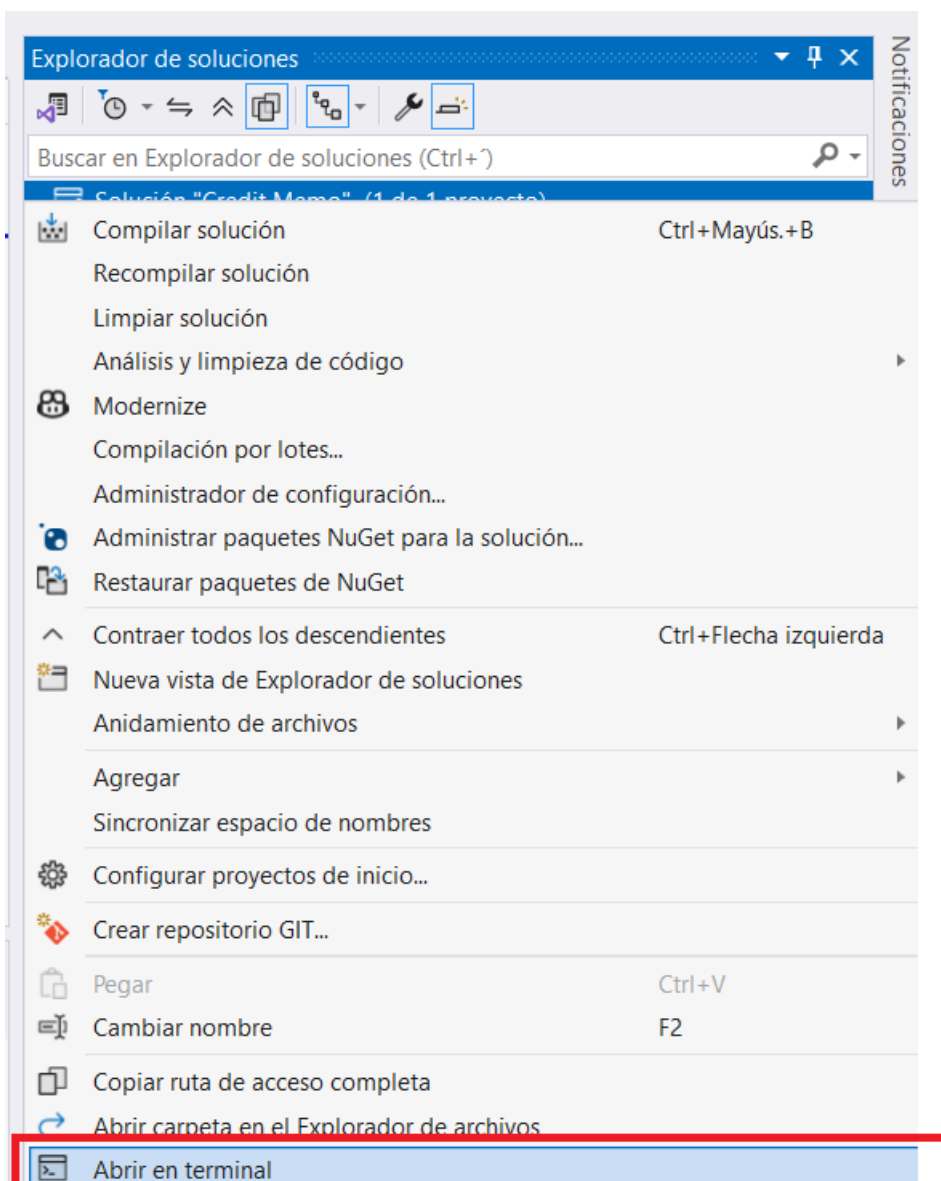
-Claves primarias y foráneas

-Relaciones (uno a muchos, muchos a muchos, etc.)

Resultado: Se genera una clase como YYYYMMDDHHMMSS_InitialCreate.cs con los métodos Up() y Down().



No confundir, Shell con consola de paquetes



Shorts-Grabación

Hay Cursos de Tecnología, Entity Framework desde la página web:

<https://theharbour.ltd>

Por ejemplo, En en el curso Entity Framework se realizará una Migración de una tabla a través de un comando: Comando:Scaffold-DbContext y se ejecutarán las operaciones correspondientes para la visualización de los datos.

Hay 4 clases en el proyecto (Web) a ser consideradas para la creación, si se quiere realizar una aplicación Web Asp.NET core

1)Models: se agrega una clase (Crédito: Constructor) por la que se quiere ejecutar los métodos de operaciones CRUD (Create, Read, Update y Delete).

1.1) Se agrega otra clase Dto: es la clase base (Las clases base no se declaran a quién heredan). El Patrón es el Master Contralador context.

1.2) Se agrega otra clase ResponseDto: se usan para transportar datos desde el backend al frontend. Es como que no tengo código que escribir.

2)Data: se agrega una clase al proyecto que es Herencia del AppDbContext (es una clase entity Framework). Donde el DBset(Credito), representa una tabla a ser considerada para las operaciones CRUD

Me permite hacer:

- Maneja la conexión a la base de datos
- Permite consultar datos
- Permite insertar, actualizar y eliminar registros
- Instalación de los Paquetes Nuggets para que funcionen AppDbContext:

1. MicrosoftEntityFrameworkCore
2. MicrosoftEntityFrameworkCore. Tools
3. MicrosoftEntityFrameworkCore.Sql

3)Clase Program, Clase Principal: Inyector de dependencia (Métodos de implementación del patrón), con operaciones lambda. Se usa este Patron Builder services.

4) Funcionalidad del Proyecto está dentro del Controllers. Ejecución del error de ResponseDto, que se usaba para transportar datos desde el backend al frontend. Se Usó un patrón de interfaz (**IEnumerable**), este lo usa en la clase principal y lo implementa en el controllers

-Inconvenientes, la creación del Proyecto en una carpeta, no se recomienda modificarla.

Ejemplos

1. Extracción –shop id credit name Currency exchange Notice do not provide a refund for change of mind account Number name surname Quantity GST Free—Tax return Origen Destino Vendor date rate destiny rate commission	2. Extracción Bancaria -Mantenimiento mensual -Extracto -Frecuencia de extracción que puede ser cuatrimestral -Comisión por operación
---	---

Se espera una app, de Delivery

Patrón de estado (Observer)

1. Objetos abstractos, Subject y Observer, y dos objetos concretos, ConcreteSubject y ConcreteObserver, que heredan los atributos de los objetos abstractos relacionados.
2. El estado a visualizar es mantenido en ConcreteSubject, que también hereda las operaciones de Subject permitiéndole añadir y eliminar Observa y enviar una notificación cuando el estado ha cambiado.
3. Objetivo del Patrón:
Separa la vista del estado de un objeto del objeto mismo y permite proporcionar vistas alternativas. Cuando el estado del objeto cambia, todas las vistas son ratificadas y actualizadas de forma automática para reflejar el cambia.
4. Los objetos que son herencia, se diagrama solo nombre y atributos, ¿y no métodos?
Porqué es como en el libro, aparece solo con dos celdas la herencia.
5. El objeto ConcreteObserver mantiene una copia del estado de ConcreteSubject e implemento la interfaz Update 0 de Observer, que permite guardar estas copias en el mismo momento.FJ objeto ConcreteObserverf visualiza automáticamente su estado -esto no es generalmente una operación de interfaz.

Diagrama

Dentro del proyecto web de api, las Carpetas: Models (métodos Concreto), Data (métodos abstractos)

1) Subject(Método Abstracto)

Person (Subject)
#Notify

2) ConcreteSubject (Método concreto)

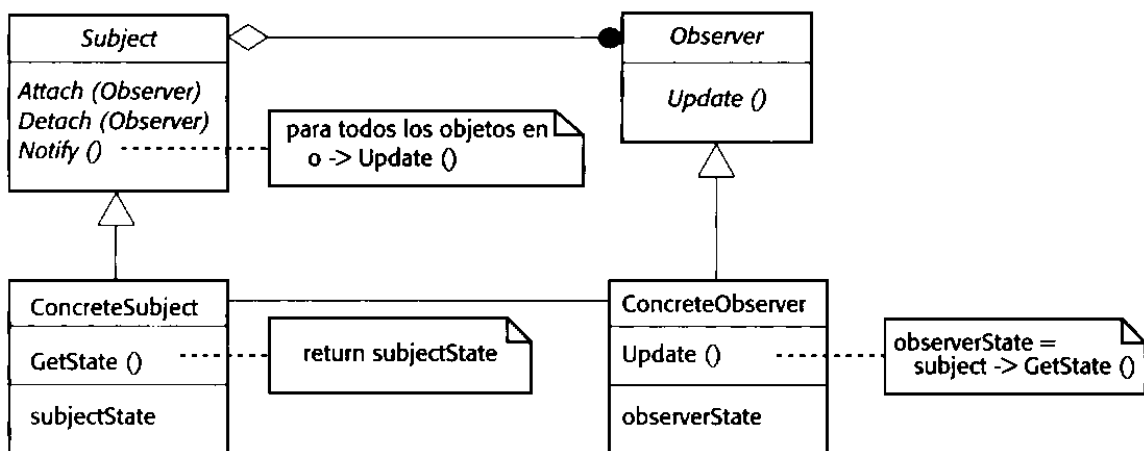
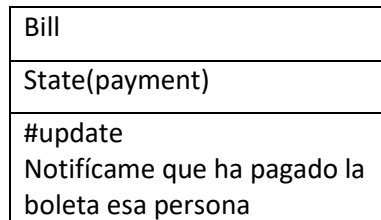
Cliente (Concrete subject)
SubjectState()

3) ConcreteObserver(Método Concreto), este lleva paréntesis, son los parámetros que está declarado en la carpeta controllers, declaración de las clases.

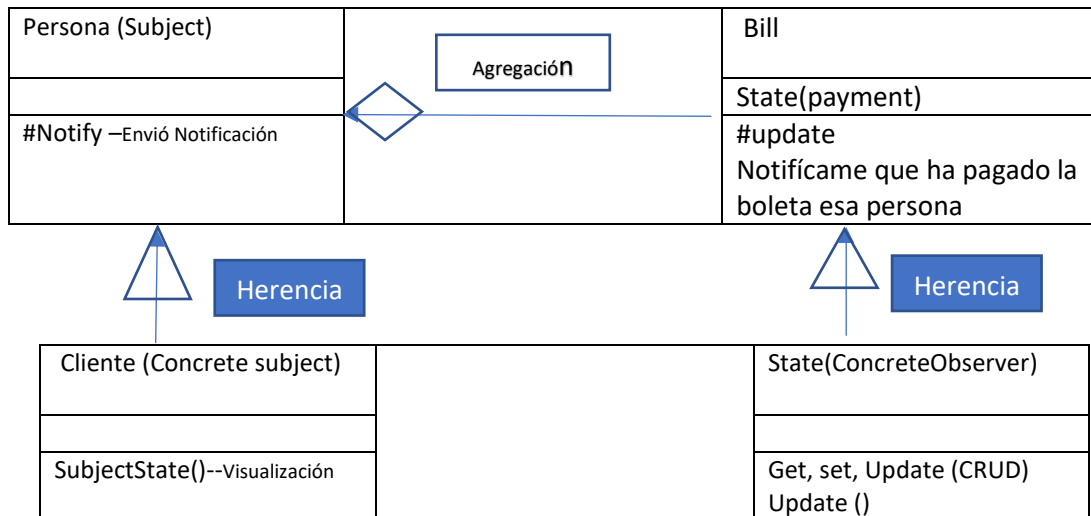
State(ConcreteObserver)
Get, set, Update (CRUD) Update()

Nota: Funcionalidad del Producto, es como que vos podes rechazar la orden porque esta defectuoso, entonces ahí si podemos usar un estado como patrón, con esta terminología: Es un término relacionado al delivery, Commercial Off-The-Shelf(COTS)

4) Observer(Método abstracto)



Relaciones



Nota:

ConcreteSubject(#Notify) es copia de los estados, porque es herencia, relacionado al observer (#update), me actualiza.

Comando: Add-Migration<nombre>	Comando:Scaffold-DbContext
Función: Genera un archivo de migración en la carpeta Migrations. Estructura de migración, estructura física -Tablas -Columnas -Tipos de datos -Claves primarias y foráneas -Relaciones (uno a muchos, muchos a muchos, etc.)	Función: genera el modelo de C# a partir de una base de datos existente. Representación lógica. Toma una base de datos ya creada (por ejemplo, SQL Server, MySQL, etc.) y genera automáticamente: Las clases de entidad (.cs)
Resultado: Se genera una clase como YYYYMMDDHHMMSS_InitialCreate.cs con los métodos Up() y Down().	Resultado: Crea un conjunto de clases que representan las tablas, relaciones y propiedades según la estructura de la base de datos.
Este comando no toca la base de datos todavía, solo genera el código de migración	Al crear una base de cero, si agregas una nueva entidad y quieres agregarla al modelo, debes ejecutar el comando anterior de migración.
Comando: Update-Database Función: Ejecuta el código SQL de las migraciones generadas con Add-Migration para modificar la base de datos.	
Resultado: EF Core actualiza la base de datos real (crea tablas, agrega columnas, etc.) y marca la migración como aplicada en la tabla interna __EFMigrationsHistory.	
Se puede hacer un restore de la base	

Ingreso y abro un nuevo proyecto, aplicación web de ASP net Core. Son Clases objetos, nada de código con Entity Framework.

Definición:

Entity Framework (EF) es un **ORM** (*Object-Relational Mapper*) desarrollado por Microsoft para .NET que permite trabajar con bases de datos usando **objetos y clases** en lugar de sentencias SQL directas.

Pasos:

- 1-Crea una carpeta **Models**, donde va una clase
- 2-Administrar los paquetes **Nuggets**, desde la opción del directorio del proyecto principal click derecho. Hay que agregar 3 componentes.
 - ✓ MicrosoftEntityFrameworkCore
 - ✓ MicrosoftEntityFrameworkCore. Tools
 - ✓ MicrosoftEntityFrameworkCore.Sql
- 3-Agregar una nueva **Clase**, que se llame **AppDbContext**, el cual hereda de DbContext
- 4-Agregar otra **Clase Dto**: No tiene lógica comunica los datos a la Api, a través del formato Json.
- 5-Agregar otra **clase ResponseDto**, no sirve como respuesta a todas las solicitudes que enviemos.
- 6- Crear el string de conexión, nos vamos al appsettings.json, abro el Management studio, y copio la conexión del server name(DESKTOP-9QN799I)
- 7-Insertar el string de conexión en Programs.cs, en nuestro inyector de dependencia.
- 8-Realizar la Migración, que se hace la consola de administración de paquetes.

Comandos

- 1-Add-MigrationDB,
- 2-Update-Database

-Crea una tabla en la base de dato

-_Agrego por cada migración una letra distinta de DB
--

9-Controladoras, donde se van a realizar los métodos http. Para eso Eliminar WeatherForecast.cs, y dentro de la carpeta de controladores también elimino.

10-En la clase de controllers, que se creó como una clase vacía, se agrega los métodos Get, Post.

```
IEnumerable<Credit> Credits = _context.Credits.ToList();
```

Se trabaja con la tabla migrada, que en este caso es Credits y la clase se llama Credit

11-Para setear la tabla que se migro que contenga datos, en la base de datos.

```
SET IDENTITY_INSERT Credits ON;
```

Después se podrá insertar un registro y probar el sistema con la función get, en la tabla Credits

12-

Se trabajo con el método post, y se probo de insertar todos los campos, pero estaba bloqueada la tabla se corrió el comando de nuevo SET IDENTITY_INSERT Credits ON; y se saco el primer atributo del campo para poder ejecutar el post, se probo en Postman y se tenia que configurar el certificado.

13- Ayuda

Fn +tecla de corchete
Fn+tecla de llave
Lambda es igual y mayor que (=>)
https://localhost:7201/swagger/index.html
No estaba el ejecutable del proyecto se corrió en la consola un comando. C:\Users\Usuario\Documents\Documents\Worksheet\ProyectoCredito\Credito\bin\Debug\net8.0 dotnet build
15-se puede crear en la clase program, cambiar el nombre, ejemplo api. <pre>app.UseSwaggerUI(c => { c.SwaggerEndpoint("/swagger/v1/swagger.json", "api"); });</pre>
Creacion de Base-2 video Comando:Scaffold-DbContext <u>Opciones por consola:</u> "Data source=(DESKTOP-9QN799I);Database=sa;Trusted_conncetion=True;TrustServerCertificate=True"; Data Source=DESKTOP-9QN799I;Initial Catalog=master;Integrated Security=True;Encrypt=False

14- el DbSet debe contener la tabla de la base de dato,
es una clase que esta en la carpeta Data

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options) { }
    //El dbSet, es la tabla de la base de datos
    public DbSet<Credit> Credits {get; set;}
}
```

Testeo en Postman

Prueba de caja negra”, porque tú envías solicitudes y validas respuestas sin ver el código interno.

-Pruebas de integración (en algunos casos)

1. Prueba de Caja Negra (Black Box Testing)

¿Qué es?

Es un tipo de prueba donde no importa cómo funciona el sistema internamente, solo se evalúa:

Lo que entra (request)

Lo que sale (response)

Características:

No necesitas ver el código, El “código interno” no es el código de Postman

Solo pruebas la funcionalidad desde afuera

Se enfoca en el comportamiento del sistema

Pruebas de integración

Contenido

Prueba	2
Migración	2
Salidas.....	2
Ejecución	3
Postman	4

Prueba

Tabla: CreditMemo

Explorador de objetos

Conectar

Tablas externas

Tablas de grafos

dbo_EFMigrationsHistory

dbo_Account

dbo_Articulos_Despatchados

dbo_Articulos_Pedidos

dbo_Client

dbo_Cliente

dbo_CreditMemo

Columnas

Id_Credit (PK, numeric(18,0), No NULL)

Name (nvarchar(50), No NULL)

Notice (nvarchar(50), No NULL)

DOB (FK, numeric(18,0), No NULL)

Passport (FK, nvarchar(50), No NULL)

Quantity (numeric(18,0), No NULL)

GST (nvarchar(50), No NULL)

Id_Origin (FK, numeric(18,0), No NULL)

Id_Vendor (numeric(18,0), No NULL)

Id_Destiny (FK, numeric(18,0), No NULL)

Claves

Restricciones

Desencadenadores

Indices

Estadísticas

SQLQuery1.sql - Usuario (73)

```
1 SELECT TOP (1000) [Id_Credit]
2 ,
3 [Name]
4 ,
5 [Notice]
6 ,
7 [DOB]
8 ,
9 [Passport]
10 ,
11 [Quantity]
12 ,
13 [GST]
14 ,
15 [Id_Origin]
16 ,
17 [Id_Vendor]
18 ,
19 [Id_Destiny]
20 FROM [master].[dbo].[CreditMemo]
```

100 %

No se encontraron problemas.

Linea: 12 Caracter: 3 SPC

Resultados

Messages

Id_Credit	Name	Notice	DOB	Passport	Quantity	GST	Id_Origin	Id_Vendor	Id_Destiny
1	1	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1

Migración

Tabla: Credits, se puede manipular, la otra no.

Explorador de objetos

Conectar

dbo_CreditMemo

Columnas

Id_Credit (PK, numeric(18,0), No NULL)

Name (nvarchar(50), No NULL)

Notice (nvarchar(50), No NULL)

DOB (FK, numeric(18,0), No NULL)

Passport (FK, nvarchar(50), No NULL)

Quantity (numeric(18,0), No NULL)

GST (nvarchar(50), No NULL)

Id_Origin (FK, numeric(18,0), No NULL)

Id_Vendor (numeric(18,0), No NULL)

Id_Destiny (FK, numeric(18,0), No NULL)

Claves

Restricciones

Desencadenadores

Indices

Estadísticas

dboCredits

Columnas

Id_Credit (PK, int, No NULL)

Name (nvarchar(max), No NULL)

Notice (nvarchar(max), No NULL)

DOB (int, No NULL)

Passport (nvarchar(max), No NULL)

Quantity (int, No NULL)

GST (nvarchar(max), No NULL)

Id_Origin (int, No NULL)

Id_Vendor (int, No NULL)

SQLQuery2.sql - Usuario (59)

SQLQuery1.sql - Usuario (73)

```
1 SELECT TOP (1000) [Id_Credit]
2 ,
3 [Name]
4 ,
5 [Notice]
6 ,
7 [DOB]
8 ,
9 [Passport]
10 ,
11 [Quantity]
12 ,
13 [GST]
14 ,
15 [Id_Origin]
16 ,
17 [Id_Vendor]
18 ,
19 [Id_Destiny]
20 FROM [master].[dbo].[Credits]
```

100 %

No se encontraron problemas.

Linea: 1 Caracter: 1 SPC

Resultados

Messages

Id_Credit	Name	Notice	DOB	Passport	Quantity	GST	Id_Origin	Id_Vendor	Id_Destiny
1	1	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1
2	2	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1
3	3	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1
4	4	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	3500	Free	1	1
5	5	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	100	Free	1	1

Salidas

1	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1	1
2	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1	1
3	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	500	Free	1	1	1
4	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	3500	Free	1	1	1
5	Currency exchange	do not provide a refund for change of mind	26963759	AA1907369	100	Free	1	1	1

Ejecución desde el

Code

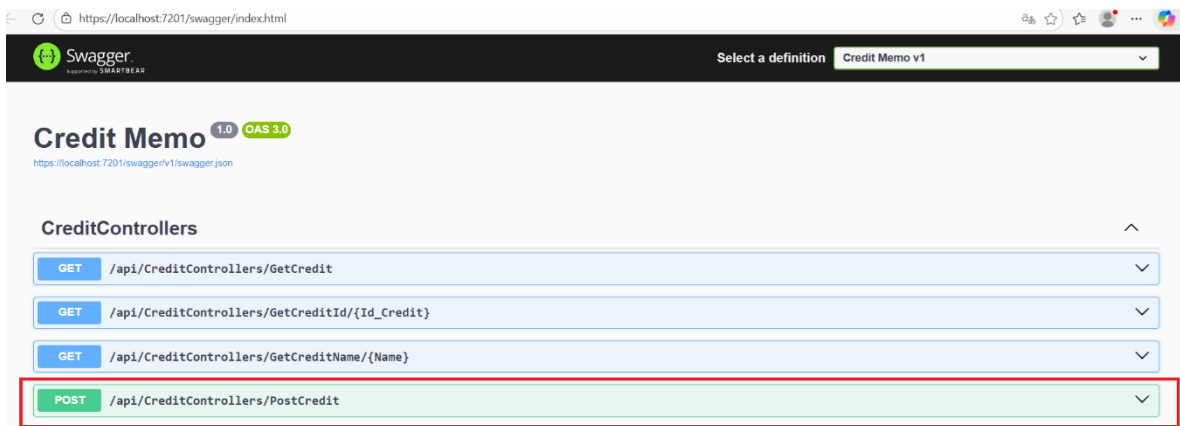
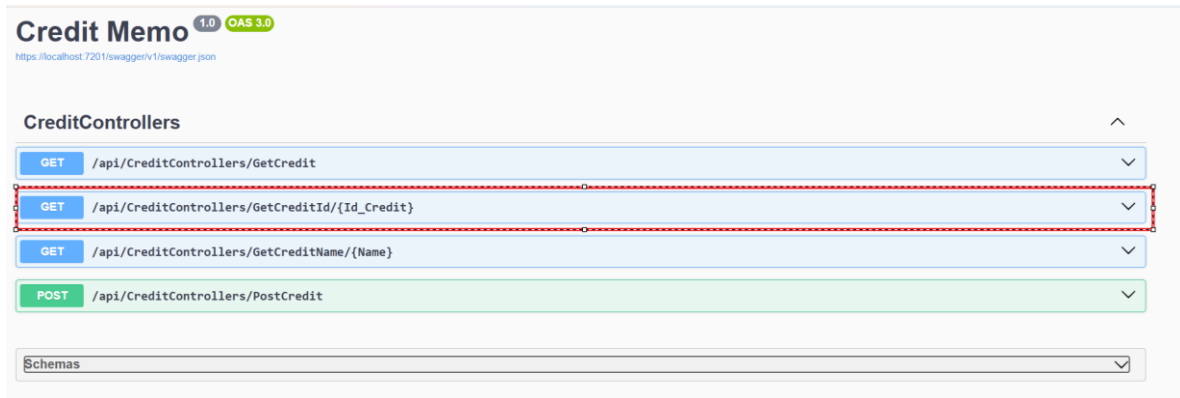
Description

200

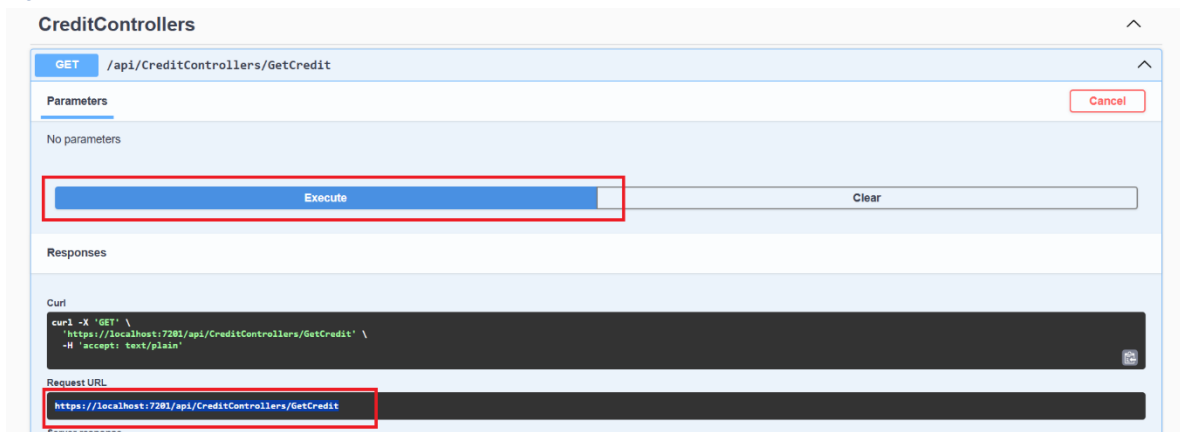
OK

Media type

text/plain



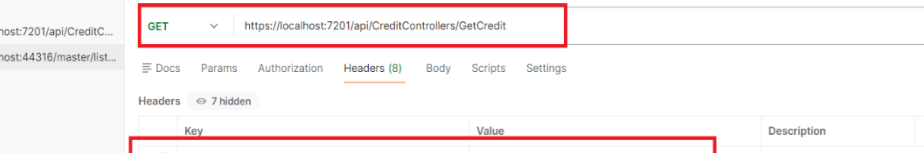
Ejecución



Pego en el browser la url

<https://localhost:7201/api/CreditControllers/GetCredit>

Postman



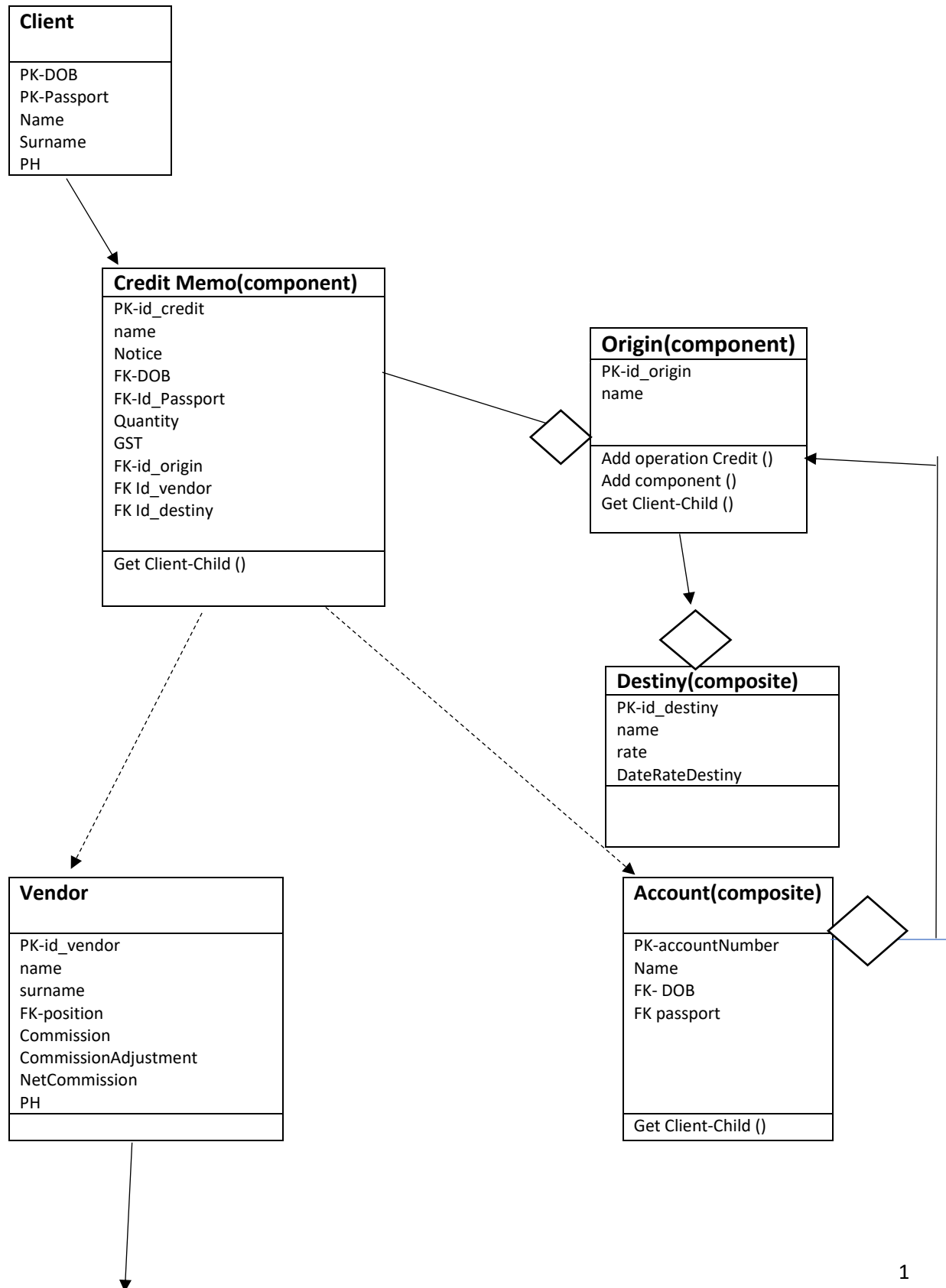
The screenshot shows the Postman interface for a GET request to `https://localhost:7201/api/CreditControllers/GetCredit`. The 'Headers' tab is selected, displaying a table with one header: 'Accept' with the value 'application/json'. The 'Body' tab is also visible, showing a JSON response with a 200 OK status.

Key	Value	Description
Accept	application/json	

200 OK • 751 ms • 1.19 KB

```
{
  "data": [
    {
      "id_Credit": 1,
      "name": "Currency exchange",
      "text": "Do not provide a refund for change of mind"
    }
  ]
}
```

Composite Pattern y herencia (Interface Realization)



Position
PK Id_position
Position

Composite pattern

Structure:

